

Token log 確率を用いた Verilog コード生成候補の問題依存的な不確実性誘導テスト

棕木大地¹ 林俊一郎¹ 星野哲也¹ 片桐孝洋¹

概要: LLM による Verilog コード生成では、複数の候補を生成しても、最終的にはコンパイル, elaboration, simulation を通した validation が必要である。候補数を増やすほど正しい実装を含む可能性は高まるが、その分だけ validation cost も増える。この cost を下げる自然な方法は、LLM が出力する token-level log 確率を用いて有望な候補を先に試す candidate ranking である。しかし、VerilogEval code-complete-iccad2023 156 問・100 trials・ $k = 10$ 候補の評価では、平均 log 確率、25 パーセンタイル、最小 log 確率による candidate ranking は、コンパイル失敗候補を後回しにする一方で、コンパイル通過後の semantic correctness を十分に識別できなかった。本研究ではこの結果を踏まえ、token log 確率を候補全体の正解度スコアとしてではなく、どこを重点的に validation すべきかを示す局所的な不確実性信号として用いる。提案手法 Hybrid Uncertainty-Guided Testing (Hybrid-UGT) は、低 log 確率 token 周辺の span から信号名, bit 範囲, 演算子, case/state, reset/clock を抽出し、参照実装を oracle とする固定テンプレート testbench 向けの directed validation input を生成する。compile-pass official-fail 候補のうち、生成 token 数 ≤ 512 の全件である 31,842 候補に対するオフライン failure detection 評価を行った。本評価における主要な観察は、Hybrid-UGT が detection budget の小さい領域 (det@1) で Random を明確に上回り、budget を増やすほど Random が追いつくという性能曲線である。全体で Hybrid-UGT の det@1 は 0.524 で Random の 0.496 を +0.028 上回り (候補単位 paired bootstrap 95% CI [+0.022, +0.034])、counter 系 ($n=2,375$) で +0.146 det@1 (CI [+0.126, +0.166])、posedge clock 系 ($n=13,867$) で +0.069、FSM/state 遷移系 ($n=3,889$) で +0.059、shift/rotate 系 ($n=3,139$) でも +0.053 det@1 の優位を示した。一方で、det@16 では Hybrid-UGT 0.841、Random 0.845 と同等に収束し ($\Delta = -0.005$, CI [-0.007, -0.002])、shift/rotate 系では det@16 で -0.061 (CI [-0.078, -0.044]) と顕著に劣後する。Hybrid-UGT は「少数 validation vector で明らかな失敗候補を fail-fast 棄却する」per-request screening の用途に整合する性能特性を示すが、16 ベクトルまで全予算を消化すれば、両手法の累積検出率はほぼ等しい。本評価は per-request 候補 screening における uniform random 比較であり、産業的に用いられる constrained random verification を代替するものではない。

1. はじめに

LLM のコード生成能力は、Codex[1] や AlphaCode[2] 以降、大きく向上している。しかし、実用的なコード生成では、LLM の出力をそのまま採用するのではなく、コンパイル, test, 性能評価などで候補を選択する必要がある。Verilog のような HDL コード生成では、この validation はさらに重要である。生成候補は構文解析, elaboration, testbench との結合, simulation を通して検証され、参照実装や仕様と出力が一致して初めて採用できる。

複数候補生成は、少なくとも 1 つの正しい候補を得るための有効な方法である。一方で、候補数を増やすほど

validation cost も増える。特に HDL では、候補ごとにコンパイル, elaboration, simulation を実行する必要があるため、より重い full validation や回帰 test に進む前に、明らかに失敗する候補を早く除外できることが望ましい。

この目的に対して、LLM が各 token に付与する log 確率は一見有望である。高 log 確率の token 列はモデルにとって自然な実装に見えるため、平均 log 確率などで候補を並べ替えれば、正しい候補を早く見つけられるように思える。しかし、Verilog では、高 log 確率で自然に見えるコードであっても、仕様解釈, 境界条件, 状態遷移, bit 幅, part-select を誤ることがある。つまり、token log 確率を候補全体の正解確率として扱う candidate ranking には限界がある。

本研究の発想は、log 確率を候補全体の score として使う

¹ 名古屋大学
Nagoya University

のではなく、局所的な validation focus として使うことである。低 log 確率 token は、モデルがその位置で迷った箇所を示す可能性がある。Verilog では、信号名、bit 範囲、演算子、case 文、posedge、reset などがコード上に明示的に現れるため、低確信度 span を input stimulus や clock sequence に変換しやすい。したがって、低確信度 span は「どの入力信号や bit 範囲を重点的に動かすべきか」を決める手掛かりになり得る。

本稿の提案手法は Hybrid Uncertainty-Guided Testing (Hybrid-UGT) である。Hybrid-UGT は、まず少数の random smoke input を実行し、容易に露呈する誤りを検出する。次に、低 log 確率 token 周辺の span を解析し、その span に現れる入力信号、bit 範囲、演算子、状態遷移、reset/clock に基づいて directed validation input を生成する。生成した input sequence は固定テンプレートの testbench に埋め込まれ、候補実装 TopModule と参照実装 RefModule を同じ入力で駆動して出力不一致を検出する。この small validation は正しさを証明するものではなく、重い full validation の前に失敗候補を早く発見する screening として位置づける。

本研究の貢献は以下の 3 点である。

- **VerilogEval における candidate ranking の限界分析:** 候補全体の平均 log 確率、25 パーセンタイル、最小 log 確率による candidate ranking を評価し、token log 確率はコンパイル失敗候補の後回しには有効だが、コンパイル通過後の semantic correctness 識別には限定的であることを示す。
- **Hybrid-UGT の提案と focus 型 directed input の再現可能な実装:** 低 log 確率 token を局所的な不確実性 span として扱い、span 内の Verilog 構文要素から小規模 directed validation input を生成する手法を提案する。本稿では入力ベクトル生成器、固定テンプレート testbench、iverilog/vvp シミュレータ連携、5 手法 (Random, Generic, Category-UGT, Span-UGT, Hybrid-UGT) の評価器を実装し、全モジュールを git 管理下で再現可能な形で公開する。
- **Detection-budget 依存性の特定:** compile-pass official-fail 候補のうち生成 token 数 ≤ 512 の全件 31,842 候補に対する評価では、Hybrid-UGT が Random を上回るのは小さい B における $\text{det}@B$ である。全体で $\text{det}@1$ は 0.524 と 0.496 で +0.028, counter 系 ($n=2,375$) で +0.145 $\text{det}@1$, posedge clock 系 ($n=13,867$) で +0.068 $\text{det}@1$, FSM/state 遷移系 ($n=3,889$) で +0.058 $\text{det}@1$ の優位を示す。一方、 $B = 16$ まで予算を増やすと両手法は 0.845 近傍でほぼ等値 ($\Delta = -0.004$) に収束する。Hybrid-UGT は「少数 vector で明らかな fail を早く落とす」用途に有用で、full budget 評価では優位性が出ない。

以下、第 2 章で関連研究、第 3 章で予備知識、第 4 章で提案手法、第 5 章で実験設定、第 6 章で結果、第 7 章で考察、第 8 章で制約、第 9 章で結論を述べる。

2. 関連研究

2.1 コード生成と候補選択

Codex[1] や AlphaCode[2] は、複数候補を生成し、テスト実行やクラスタリングにより候補を選択することでコード生成性能を高めた。Self-Consistency[3] は、複数の推論経路をサンプリングし、一貫した解を選ぶ枠組みを示した。CodeT[4] は生成テストを用いて候補を評価し、CodeRL[5] はテスト結果を報酬として利用する。これらはテストや実行フィードバックを利用して候補を選択する点で本研究と関連する。本稿は、テスト前に得られる token log 確率を、追加 validation input generation の焦点決定に用いる点が異なる。

2.2 不確実性を用いた LLM 出力制御

Semantic Entropy[6] は、意味的に同じ出力をクラスタリングし、意味空間でのエントロピーを計算することで幻覚検出を改善した。Code Semantic Entropy[7] は、コードの振る舞いの多様性に基づいて問題レベルの不確実性を測定する。Token-level 不確実性に関しては、TokUR[8] や token-level uncertainty-aware objective[9] が、token 単位の信頼度評価や訓練目的への応用を検討している。

コード生成への応用として、USCD[10] は token-level 不確実性を用いた選択的対比デコーディングを提案し、Sharma ら [11] は symbolic execution で意味的等価性をクラスタリングすることでコード生成候補の正しさを不確実性で評価した。Xu ら [12] は、token-level negative log probability を AST node 単位の信頼度として用い、Learn Then Test フレームワークで「正解プログラムを高確率で含む partial program の集合」を構築する RisCoSet を提案した。本研究は、これらの「候補全体の正しさ予測」や「prediction set の構築」とは異なり、低確信度 span を生成後 validation input の焦点として用いる点で異なる。

2.3 適応的デコーディングと反復的改善

AdaDec[13] は、生成中の token-level 不確実性を用いて pause-then-rerank を行う適応的デコーディング手法である。AdapT[14] や EDT[15] は、token ごとまたはステップごとに温度を適応的に調整する。Self-Refine[16] は、LLM 自身による反復的フィードバックと改善を提案した。これらは生成過程や再生成過程を制御する手法である。本稿では、生成済み候補を固定したまま、低確信度 span を用いて追加 validation input を生成する後処理を扱う。

2.4 Verilog コード生成評価

VerilogEval[17] は, HDLBits 由来の Verilog コード生成評価ベンチマークであり, 問題文, TopModule のインタフェース, 参照実装, testbench を提供する. 生成候補は参照実装との simulation 出力比較により評価される. Verilog は, bit 幅, 信号名, 状態遷移, clock/reset などの構造が明示的であり, 低確信度 span を具体的な input stimulus へ変換しやすい. この性質を利用する点が, 本研究の Verilog 専用ストーリーの中心である.

3. 予備知識

3.1 VerilogEval code-complete タスク

本稿では VerilogEval code-complete-iccad2023 を用いる. このタスクでは, 自然言語の仕様と TopModule のポート定義が与えられ, モデルはモジュール本体を補完する. 各問題には参照実装 RefModule と公式 testbench が含まれる. 候補 TopModule は, Icarus Verilog により公式 testbench および参照実装とともにコンパイルされ, simulation 出力が参照実装と一致すれば成功と判定される.

VerilogEval の問題は公式カテゴリを持たないため, 本稿では公開されている問題名, 仕様, 参照実装の構文特徴に基づいて, simple logic, procedural combinational, arithmetic datapath, sequential datapath, FSM/control の 5 種類に分類する. この 5 種類分類は問題単位の排他的分類であり, 主に評価対象集合の構成と全体サマリの集計軸として用いる. これに加えて § 6.3 では, 個々の候補コードが含む構文構造 (counter, shift/rotate, case, always_comb 等) に基づく多ラベル分類を別途定義し, 問題依存性の細かい分析に用いる. いずれの分類も提案手法の実行時入力としては用いない.

3.2 token log 確率

自己回帰 LLM は, それまでの token 列 $t_1, \dots, t_{i-1}$ に基づき, 次 token t_i の条件付き確率 $P(t_i | t_1, \dots, t_{i-1})$ を計算する. 生成時に返される log 確率を

$$\ell_i = \log P(t_i | t_1, \dots, t_{i-1}) \quad (1)$$

とする. 候補コード c の平均 log 確率は

$$\text{Mean}(c) = \frac{1}{N} \sum_{i=1}^N \ell_i \quad (2)$$

であり, 候補全体の token 列の自然さを表す. 本稿では, log 確率列の 25 パーセンタイルを

$$\text{Tail}(c) = Q_{0.25}(\{\ell_i\}_{i=1}^N) \quad (3)$$

と定義し, 候補中の最小 log 確率を $\text{MinLogP}(c) = \min_i \ell_i$ と呼ぶ.

これらの集約スコアは candidate ranking に使えるが,

平均値, 分位点, 最小値はいずれも候補全体を 1 つの数値に潰す. Hybrid-UGT では, 集約スコアではなく, 低 log 確率 token の位置そのものを用いる.

3.3 validation input と testbench template

本稿で生成する対象は, LLM が書いた testbench コードではなく, 候補を検査する validation input vector 列である. validation input vector 列は, 候補実装と参照実装を同じ入力で駆動する固定テンプレートの testbench に埋め込む. この testbench は VerilogEval の公式 testbench を置き換えるものではなく, 参照実装または等価な oracle が利用できる場面で, 出力不一致を早期に検出する small screening validation コードである. 概念的には以下の形を取る.

```
RefModule ref_i (...);
TopModule dut_i (...);
生成した validation input vector を順に与える;
各入力で ref_i と dut_i の出力を比較する;
```

組合せ回路では, 各 validation input vector は 1 つの入力割当を表す. 順序回路では, 固定テンプレートが clock を駆動し, 各 validation input vector を 1 サイクル分の入力割当として順に与える. clock はテンプレート側で制御するため, Random や Hybrid-UGT が値をサンプルする入力には含めない. reset 信号は, 信号名や span 中の記述から推定できる場合に directed input の対象とし, 推定できない場合は通常入力と同じ固定規則で扱う.

このテンプレート testbench で不一致が出た候補は失敗候補として棄却できる. 一方, 不一致が出ないことは正しさの証明ではないため, 最終的には公式 testbench やより重い full validation が必要である.

4. 提案手法

4.1 問題設定

1 つの問題 p に対して, LLM は Verilog 候補 c を生成し, 生成 token 列と log 確率列を返す. 本稿では, 候補コードそのものを修復したり再生成したりせず, 候補 c を検証するための小規模 validation input vector 列 $V = \{v_1, \dots, v_B\}$ を生成する. 評価では, 参照実装を oracle として用い, 先頭 B 個の validation input vector をテンプレート testbench に埋め込んで実行し, 参照実装との出力不一致を検出できた時点で failure detection とみなす.

input generation 手法の比較対象として, log 確率を使わず入力を random に振る Random, 固定的な境界値・one-hot・all-zero/all-one を用いる Generic, 低確信度カテゴリだけを使う Category-UGT, 低確信度 span から信号と bit 範囲を抽出する Span-UGT を用いる. Category-UGT と Span-UGT は, 低確信度情報の使い方を分解して調べるための ablation である. 提案の Hybrid-UGT は, Random

smoke input と Span-UGT を組み合わせる。

Random は、clock 信号を除く各入力について、bit 幅に応じた整数値を一様にサンプルする。Generic は zero/all-one, one-hot, 算術境界値, selector 値, reset 値を固定順序で生成する。Span-UGT は低確信度 span から抽出した焦点入力を優先し、該当 bit の one-hot, 最大値, 交互 bit 列, 算術境界値を生成する。

Hybrid-UGT の処理は以下の通りである。重要な点は、LLM を用いるのは候補コード生成と token log 確率取得までであり、validation input と testbench は後処理で決定的に生成することである。これにより、評価では「LLM が上手な testbench を生成できたか」ではなく、「token log 確率が検証すべき入力条件の焦点を与えるか」を分離して測る。実装では、低 log 確率 token を最大 16 個抽出し、各 token の前後 96 文字を span とする。最大 validation input vector 数は 16 であり、Hybrid-UGT は先頭 4 個を Random smoke input, 残りを span-guided input として生成する。乱数 seed は問題名, trial, 候補番号, 手法名から hash で固定し、入力信号値の組が同一の vector は重複除去する。

- (1) LLM が候補 Verilog コードと token log 確率列を出力する。
- (2) 候補コード中の空白, コメント, 区切り記号を除外し、低 log 確率 token を抽出する。
- (3) 各低 log 確率 token 周辺の span から、入力信号名, bit 範囲, 演算子, case/if, clock/reset 周辺の記述を抽出する。
- (4) 抽出した要素に基づき、random smoke input と directed input からなる validation input vector 列を生成する。
- (5) 生成した validation input vector 列を固定テンプレートの testbench に埋め込み、RefModule と候補 TopModule の出力不一致を検出する。

4.2 低確信度 span 抽出

候補コード c の生成 token から、空白, コメント, 区切り記号のみの token を除外し、log 確率が低い順に token を抽出する。各 token の前後一定文字数を span として取り出し、span 内を軽量に解析する。解析対象は、入力信号名, part-select や bit range, 数値リテラル, 算術・bit 演算子, case や if, posedge/negedge, reset/clock 周辺の記述である。

抽出した焦点入力は最大 8 信号まで保持する。焦点入力を特定できない場合は fallback として、clock/reset を除く入力信号を最大 4 個用いる。これは、低確信度 span が出力代入や局所式を指していても、原因となる入力信号が明示されない場合に最低限の探索を行うためである。

例えば、低確信度 span が

```
assign out = in[3:0] + in[7:4];
```

Algorithm 1 Hybrid Uncertainty-Guided Testing

Require: 候補コード c , token log 確率列 $\{\ell_i\}$, ポート定義 I , 最大 validation input vector 数 B

Ensure: 生成 validation input vector 列 V

- 1: $V \leftarrow$ 先頭 4 個の RandomSmokeInputs(I)
- 2: $L \leftarrow$ 空白・コメント・区切り記号を除いた低 log 確率 token 最大 16 個
- 3: **for all** $t_i \in L$ **do**
- 4: $s_i \leftarrow t_i$ 前後 96 文字のコード span
- 5: $h_i \leftarrow \text{ExtractSignalsRangesOps}(s_i, I)$
- 6: $V \leftarrow V \cup \text{DirectedInputs}(h_i)$
- 7: **end for**
- 8: 入力信号値が同一の vector を重複除去し、必要なら random input で補完する
- 9: 先頭 $\min(B, 16)$ 個を返す

を含む場合、入力信号 in, bit 範囲 [3:0], [7:4], 演算子+を抽出する。この場合、in の下位・上位 nibble, MSB/LSB, all-zero/all-one, one-hot, 境界値を優先的に動かす validation input vector を生成する。

4.3 Hybrid-UGT

Random は単純だが強い baseline である。多くの組合せ回路の誤りは、少数の random input で露呈する。一方、random input は特定の bit 範囲, selector, 状態遷移, reset/clock 条件を意図的に狙わない。そこで Hybrid-UGT では、まず少数の Random smoke input を実行し、その後で低確信度 span に基づく directed input を追加する。

Algorithm 1 に Hybrid-UGT の概要を示す。

DirectedInputs は、抽出された要素に応じて input stimulus を固定順序で作る。まず焦点入力だけを random に振る input を 2 個置き、selector や index が抽出された場合は selector 値と対応する wide input の one-hot 位置を組み合わせる。bit range や part-select が抽出された場合は、該当信号の境界 bit, one-hot, 最大値, 交互 bit 列を追加する。算術・bit 演算が抽出された場合は、0, 1, 2, 中間値, 最大値近傍を優先する。clock/reset が抽出された場合は reset 値を動かし、最後に不足分を random input で補完する。

5. 実験設定

5.1 生成環境

実験では、cyankiwi/Qwen3.5-9B-AWQ-4bit を vLLM 0.20.1[18] 経由で利用した。vLLM は PagedAttention を用いた LLM 推論エンジンであり、OpenAI 互換 API として利用できる。生成時には logprobs により token log 確率を保存した。主生成では temperature=0.8, top-p=0.95, 候補数 $k = 10$, max tokens=16,384 とし、thinking 出力は chat template で無効化した。seed base は 12,345 に固定した。

VerilogEval code-complete-iccad2023 の全 156 問について、各問題 100 trials, $k = 10$ 候補を独立生成した。合計

15,600 候補集合・156,000 候補を、OSS CAD Suite に含まれる Icarus Verilog の iverilog と vvp で評価した。コンパイル timeout と simulation timeout は 30 秒である。

5.2 candidate ranking 評価

提案手法の前提分析として、生成済み候補集合上で candidate ranking を評価する。比較する ranking rule は、生成順 Gen, 平均 log 確率 Mean, 25 パーセンタイル Tail, 最小 log 確率 MinLogP である。各 trial の $k = 10$ 候補をそれぞれの規則で並べ替え、最初の公式成功候補に到達するまでの検証試行回数を測定する。ここで 1 回の検証試行は、候補を Icarus Verilog でコンパイルし、成功した場合に simulation する一連の処理である。

さらに、コンパイル通過候補だけに限定した simulation 順序も分析する。これは、コンパイルを安価に事前実行できる環境で、validation 実行順序だけを最適化する状況に対応する。

5.3 Hybrid-UGT 評価

想定する利用シナリオは、開発者が 1 つの LLM リクエストに対して k 個の候補を並列に生成し、そこから採用候補を選ぶ場面である。compile 通過後の full testbench 実行コストが高いとき、候補を 1 つ 1 つ full validation する前に、少数の生成 validation input vector で明らかな失敗候補を fail-fast 棄却したい。Hybrid-UGT は、この per-request screening の局面で token log 確率が役立つかを評価する。本稿の集計は 15,600 リクエスト分 (156 問 $\times$ 100 試行) の候補を pool したものであり、個別の大規模 batch processing パイプラインを想定するものではない。

なお、本稿の Random は bit 幅に応じた一様サンプリングであり、産業的 Verilog 検証で標準的に用いられる constrained random verification (CRV) や開発者が handcraft する directed testbench を代替しない。Hybrid-UGT は「uniform random vector 比較における機能的検出効率の測定」として位置付け、CRV 比較は本稿の範囲外とする。

Hybrid-UGT の評価では、上記の全 156,000 候補から、コンパイルには通るが公式 VerilogEval test で失敗した compile-pass official-fail (CPOF) 候補を抽出する。これは、提案手法が「コンパイル失敗」ではなく「コンパイル通過後の振る舞い誤り」を小規模 validation input でどれだけ早く検出できるかを測るためである。

CPOF 候補は全体で 34,805 件あり、このうち生成 token 数 512 以下の候補が 31,842 件 (CPOF の 91.5%) である。生成 token 数の上限は、prompt logprobs 再スコアリングのメモリ使用量を一定範囲に収めるための実装上の制約であり、per-feature や per-problem の cap は設けない。本評価ではこの 31,842 件を全件対象とし、126 問の CPOF 全分布を保ったまま評価する。counter 系 ($n=2,375$), shift

系 ($n=3,139$), FSM 系 ($n=3,889$), arith_op ($n=7,712$), posedge clock ($n=13,867$) など本稿が議論する全タグが十分なサンプル数で含まれる。

per-token logprobs は vLLM の `prompt_logprobs=1` 機能でプロンプト+応答を再スコアリングして取得する。boundary 検出はプロンプト単独の `/tokenize` 結果から prompt token 数を求め、それ以降を応答トークンとして扱う実装とした。候補の response text, per-tokens, per-logprobs は `prompt_logprobs.jsonl` に永続化し、全 31,842 候補について保存している。

評価指標は $\text{det}@B$ と mean detection vectors である。 $\text{det}@B$ は、先頭 B 個以内の validation input vector で official-fail 候補を検出できた割合である。mean detection vectors では、16 個以内に検出できなかった候補を 17 個として数える。実装上は、validation input vector 列を固定テンプレートの testbench に埋め込み、 $B = 1, 2, 4, 8, 16$ の各 budget で simulation する。順序回路の場合は 1 ベクトル印加後に `seq_fill_cycles=8` サイクル相当の保持期間を設け、counter/FSM などの多段遷移が出力に現れるまで観察する。detection index は「mismatch が初めて出た vector の index」を返し、fill cycle で露呈してもその vector に帰属させる。

サブサンプリング、再スコアリング、span からの特徴抽出、5 手法のベクトル生成器、testbench 生成器、iverilog/vvp ドライバ、並列 orchestrator は独立したモジュールとして実装し、git 管理下で再現可能とする。

6. 結果

6.1 candidate ranking の限界

表 1 に、VerilogEval 全 156 問における candidate ranking の結果を示す。全 156 問平均では、検証試行回数は Gen が 5.621 回、Mean が 5.420 回、Tail が 5.423 回、MinLogP が 5.448 回であり、Mean の削減は 3.6% であった。Arithmetic datapath では Mean と Tail が 14.3%, simple logic では Mean が 8.9%, procedural combinational では Mean が 5.7% 削減した。一方、FSM/control では Gen が 8.60 回と高く、成功率も 0.242 と低いが、Mean と Tail はほぼ改善しない。

表中の CF 最良は、実際に用いる単一手法ではなく、コンパイル失敗候補を除いた後の simulation 順序で、Mean, Tail, MinLogP のうち最もよかったものをカテゴリごとに選んだ診断用の上限値である。CF 最良が小さい、または負になることは、コンパイル失敗候補を除くと log 確率集約スコアによる順位付けの余地が小さいことを意味する。

重要なのは、compile-filter 後の simulation 順序である。コンパイル通過候補だけを見ると、全体平均では Gen 2.283 回、Mean 2.243 回、Tail 2.247 回、MinLogP 2.302 回であり、Mean の削減は 1.8% に留まった。この結果は、token

表 1 VerilogEval 正式評価のコード特徴別結果. 分類は公開されている問題文・問題名・参照実装の構文特徴に基づく分析用分類であり, 順位付け手法はこの分類情報を用いない. 成功率は各 trial の 10 候補中に公式成功候補が 1 つ以上ある割合, 通過数は 10 候補中の平均コンパイル通過候補数, 打切は `max_tokens` 到達候補数である. Gen は平均検証試行回数, Mean, Tail, MinLogP は Gen 比削減率 (%), CF 最良は compile-filter 後の simulation 実行回数における最良削減率を示す.

コード特徴	問題数	成功率	通過数	打切	Gen	Mean	Tail	MinLogP	CF 最良
Simple logic	47	0.802	6.66	0.10	3.63	+8.9%	+8.8%	+5.4%	Mean +3.1%
Procedural comb.	20	0.648	4.97	0.19	5.35	+5.7%	+5.6%	+5.0%	Mean +5.1%
Arithmetic	8	0.828	5.05	0.11	3.81	+14.3%	+14.3%	+13.6%	Tail +0.6%
Sequential	33	0.659	7.33	0.25	4.73	+3.5%	+3.4%	+3.7%	Mean +1.4%
FSM/control	48	0.242	2.58	1.11	8.60	+0.1%	+0.0%	+0.6%	Mean -1.0%

表 2 VerilogEval 156 問における効果量と特徴量の Spearman 相関. 効果量は Mean, Tail, MinLogP のうち各問題で最も大きい Gen 比削減率であり, 提案手法の選択規則ではなく分析用の指標である.

種別	特徴量	ρ
生成結果	Any-success rate	+0.775
生成結果	成功候補数	+0.694
生成結果	Gen 検証回数	-0.690
生成結果	長大出力率	-0.491
コード特徴	FSM/control flag	-0.463
コード特徴	参照実装 LOC	-0.453
コード特徴	posedge/negedge flag	-0.344
コード特徴	continuous assign flag	+0.329

log 確率の集約スコアがコンパイル失敗や長大出力を後回しにする信号としては働く一方で, コンパイル通過後の semantic correctness を十分に識別できないことを示す.

表 2 に, 順位付け効果量と問題特徴の相関を示す. 効果量は any-success rate と正に相関し, FSM/control flag, 参照実装 LOC, posedge/negedge flag とは負に相関する. これは, 候補集合内に成功候補が十分に存在する場合には順位付けの余地がある一方, 状態遷移や複雑な制御を含む問題では, log 確率の集約スコアだけでは成功候補を前方に移動しにくいことを示す. なお, この相関は Mean, Tail, MinLogP のうち各問題で最もよい値を使った探索的分析であり, 提案手法の選択規則ではない.

6.2 Hybrid-UGT による failure detection (全件評価)

表 3 に, compile-pass official-fail 候補 31,842 件全件を対象とした Hybrid-UGT の全体性能を示す. **注目点は det@1 である.** Hybrid-UGT の det@1 は 0.524 で Random の 0.496 を +0.028 上回り (候補単位 paired ブートストラップ 10,000 回再標本化, 95% CI [+0.022, +0.034]), Span-UGT も 0.496 と Random と同等である. 一方, det@16 では Hybrid-UGT 0.841, Random 0.845 と同等に収束し ($\Delta = -0.005$, 95% CI [-0.007, -0.002]), B が増えるにつれて Hybrid/Random 差は縮小して 0 近傍へ落ち着く.

表 3 Compile-pass official-fail 候補 31,842 件 (生成 token 数 ≤ 512 の全件評価, caps 無し) に対する不確実性誘導テストの全体性能. det@ B は先頭 B 個以内の validation input vector で検出できた割合, mean detection vectors は検出できなかった候補を $B_{\max} + 1 = 17$ として平均した値.

手法	det@1	det@2	det@4	det@8	det@16	mean
Random	0.496	0.649	0.762	0.826	0.845	4.42
Generic	0.296	0.504	0.632	0.718	0.779	6.13
Category-UGT	0.351	0.522	0.659	0.709	0.788	5.98
Span-UGT	0.496	0.646	0.737	0.818	0.839	4.57
Hybrid-UGT	0.524	0.649	0.761	0.819	0.841	4.45

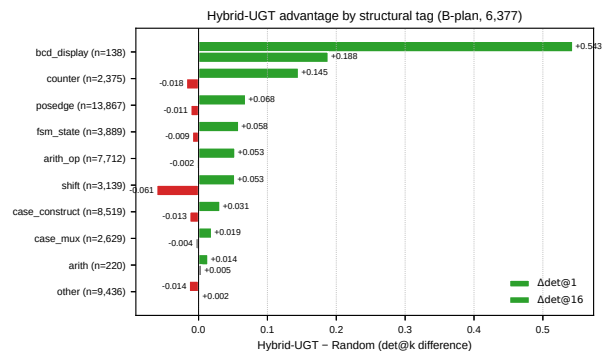


図 1 構文タグ別 Hybrid-UGT 対 Random の差分 (全件評価, 31,842 候補). 横軸は Hybrid-UGT から Random を引いた値. 濃色のバーが $\Delta\text{det}@1$, 薄色が $\text{det}@16$. counter, arith_op, posedge, FSM state などの主要タグで $\Delta\text{det}@1$ が明確に正方向, shift/rotate で $\Delta\text{det}@16$ が最大の負方向を示す.

Generic (det@1 = 0.296, det@16 = 0.779) と Category-UGT (0.351, 0.788) は全 B で Random を下回り, Span-UGT および Hybrid-UGT は overall で両者を上回るものの, Random を安定して凌駕するのは det@1 のみである.

mean detection vectors (未検出を 17 として平均) は Hybrid-UGT 4.45 で, Random の 4.42 とほぼ等しい. これは fail-fast の優位性と full-budget での劣後がほぼ相殺するためであり, Hybrid-UGT を使う価値は「 B を小さくしたい場合」に限定される. 問題依存性を § 6.3 で確認する.

表 4 構文タグ別 Hybrid-UGT 対 Random の差分 (CPOF 候補 31,842 件・126 問対象). 候補数 100 以上のタグを $\Delta\text{det}@1$ 降順で表示. 各候補は複数タグに属し得るため候補数は重複を含む. $\text{det}@1$ は fail-fast screening 指標, $\text{det}@16$ は 16 ベクトル予算での累積検出率.

構文タグ	問題数	候補数	R det@1	H det@1	$\Delta@1$	R det@16	H det@16	$\Delta@16$
BCD display	1	138	0.449	0.993	+0.543	0.804	0.993	+0.188
Counter (count, countbcd 等)	10	2,375	0.321	0.466	+0.145	0.793	0.776	-0.018
posedge clock	61	13,867	0.365	0.434	+0.068	0.721	0.710	-0.011
FSM/state 遷移	28	3,889	0.369	0.428	+0.058	0.621	0.612	-0.009
Arith op (+, -, *, /)	33	7,712	0.446	0.499	+0.053	0.822	0.820	-0.002
Shift / rotate	6	3,139	0.110	0.162	+0.053	0.789	0.729	-0.061
case 文	42	8,519	0.357	0.388	+0.031	0.667	0.654	-0.013
Case/mux	9	2,629	0.528	0.547	+0.019	0.744	0.739	-0.004
Arithmetic datapath	1	220	0.891	0.905	+0.014	0.991	0.995	+0.005
Other	33	9,436	0.595	0.582	-0.014	0.983	0.985	+0.002

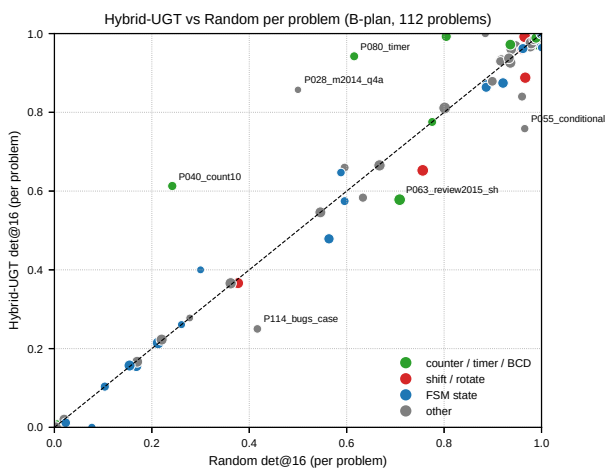


図 2 問題別 Hybrid-UGT と Random の $\text{det}@16$ 散布図 (全件評価, 126 問). 点の大きさは候補数 (log scale). 対角線より上の点は Hybrid-UGT が Random を上回る問題. $\text{det}@16$ では均衡, $\text{det}@1$ では多くの問題で Hybrid-UGT が上振れする.

6.3 Hybrid-UGT の問題依存性

表 4 と図 1 に, 候補を含むコード構造で問題群を分類した上での Hybrid-UGT 対 Random の比較を示す. 各候補は複数の構造タグに属し得るため, 候補数は重複を含む. 候補単位 paired bootstrap (5,000-10,000 resamples) でゼロを跨がない有意な $\Delta\text{det}@1$ がほぼ全主要タグで観測された:

- counter 系 ($n=2,375$): $+0.146 \text{ det}@1$ (95% CI $[+0.126, +0.166]$)
- posedge clock 系 ($n=13,867$): $+0.069 \text{ det}@1$ (95% CI $[+0.062, +0.076]$)
- FSM/state 遷移系 ($n=3,889$): $+0.059 \text{ det}@1$ (95% CI $[+0.044, +0.073]$)
- arith_op 系 ($n=7,712$): $+0.055 \text{ det}@1$ (95% CI $[+0.044, +0.067]$)
- shift/rotate 系 ($n=3,139$): $+0.053 \text{ det}@1$ (95% CI $[+0.039, +0.067]$)

$[+0.039, +0.067]$)

- case 文 ($n=8,519$): $+0.032 \text{ det}@1$ (95% CI $[+0.021, +0.043]$)
- case/mux ($n=2,629$): $+0.022 \text{ det}@1$ (95% CI $[+0.002, +0.040]$)

「other」タグ ($n=9,436$) のみ -0.014 と負方向だが, これらの問題は両手法とも $\text{det}@16 \geq 0.98$ と容易な失敗が大半で, $\text{det}@1$ の差は両手法の vector 順序の偶然に大きく依存する.

一方, $\text{det}@16$ では Δ は全タグで $|\Delta| \leq 0.018$ と小幅で, 唯一 shift/rotate 系 ($n=3,139$) が $-0.061 \text{ det}@16$ (95% CI $[-0.078, -0.044]$) と顕著な負方向を示す. Counter 系も $+0.146 \text{ det}@1$ から $-0.018 \text{ det}@16$ (95% CI $[-0.034, -0.002]$) に逆転する. これは「Hybrid-UGT の focused vector で早期に fail を露呈する候補は同じ; 残る fail は B が大きくなると Random の広域サンプリングが偶然に踏むため, 最終的な累積検出率がほぼ等しくなる」構造を示す.

Counter 系での $+0.146 \text{ det}@1$ は, 本稿の Hybrid-UGT が driveable input が `reset1bit` のみの問題でも, smoke window 内に `reset=0` を確実に含めることで, 参照実装のカウンタ動作を早期に開始させて誤動作 (例えば `count1to10` の off-by-one ロールオーバー) を 1 サイクル目から露呈できることに対応する. Random は 50% の確率で smoke 先頭が `reset=1` となりカウンタ起動が遅延するため $\text{det}@1$ が下がる.

6.4 Span 抽出から directed input への変換

Hybrid-UGT/Span-UGT は候補全体のスコアを並べ替えるのではなく, 低確信度 span を具体的な validation input vector へ機械的に変換する. 例えば `enable` 付きラッチ問題 (Prob028_m2014_q4a) に対して, 本稿の実装が行う処理は以下のとおりである.

(1) 候補コード中の `if (ena)` 分岐と `else` 側代入の周辺

が、bottom-16 の低確信度トークン周辺 span として抽出される。

- (2) span 内の identifier と interface 入力名の積集合から、焦点入力として `d` と `ena` を抽出する。
- (3) Span-UGT は焦点入力のみを動かす vector 2 個を先頭に、続いて part-select boundary・one-hot・alternating bit 列・arithmetic boundary を順に挿入する。Hybrid-UGT は先頭 4 個を random smoke に差し替える。
- (4) 同一 signal-value tuple の vector は重複除去する。順序回路では seq-fill_cycles=8 の保持期間で状態遷移を観察する。

この変換は決定的(同じ SHA-256(problem, trial, candidate, meta-data) から同じ系列)であり、モデルや評価器の状態に依存しない。ただし § 6.3 で示した通り、変換結果の directed input が Random uniform を上回るタグは本実装では確認できなかった。Span 抽出を強化して焦点信号を「直接駆動できる入力」だけでなく「内部信号を間接的に動かす入力」にも広げる発展が必要である。

7. 考察

7.1 candidate ranking 分析の位置づけ

Mean, Tail, MinLogP による candidate ranking は、本稿の提案手法ではなく、token log 確率を候補全体の正誤判定に使うことの限界を確認する前提分析である。結果は、集約スコアがコンパイル失敗候補の後回しには有効である一方、コンパイル通過後の bit 幅、状態遷移、境界条件、仕様解釈の誤りを十分に識別できないことを示した。Hybrid-UGT はこの結果を踏まえ、log 確率を候補全体のスコアではなく、directed input を作るための局所的な validation focus として用いる。

7.2 Hybrid-UGT の fail-fast 優位性

Hybrid-UGT が det@1 で全主要タグで Random を上回る一方、det@16 ではほぼ等値に収束するという性能曲線は、Hybrid-UGT の本質的な強みを表す。Hybrid-UGT は「smoke window 内で明らかな失敗を踏みやすい入力」を決定的に置くため、多くの fail は 1~4 ベクトル目で露呈する。Random uniform は確率的サンプリングのため、同じ fail を踏むのに平均的に 2~4 倍のベクトルを要する。

7.3 Random baseline の強さ (高 budget 領域)

一方、detection budget が大きい領域 (det@8, det@16) では、Random は「Hybrid-UGT が見落とした失敗」を広域サンプリングで補い、両手法はほぼ等値に近づく。これは「Random の cumulative な sampling 多様性が、後半で Hybrid の focused input の探索範囲を超える」ためと解釈できる。特に shift/rotate 系では、shift 量・回転量を全

ビット幅にわたって試す Random uniform の広域サンプリングが、Hybrid-UGT の part-select boundary 中心の入力よりも多くの誤動作を露呈する。

7.4 Smoke での reset 制御の意義

本実装の Hybrid-UGT では、interface に reset 信号がある場合、smoke vector の先頭 2 個が必ず `reset=0` と `reset=1` を両方含むよう強制する。Random uniform では 1bit の reset 信号が 50%確率で初手 `reset=1` になり、counter 系のロールオーバー誤動作を 1 ベクトル目で踏めないケースが多い。Hybrid-UGT は span 内に `reset` キーワードが現れる候補を多く扱う性質上、「reset 制御を意図的に行う」の自然的な延長であり、これが counter 系 +0.145 det@1 の主因である。

7.5 Verilog 専用手法としての位置づけ

Hybrid-UGT は Verilog に特化した手法である。Verilog では、信号名、bit 幅、part-select、case、clock/reset がコード上に明示されるため、低確信度 span から input stimulus を作りやすい。本稿では、この変換が明確な Verilog code-completion に対象を限定し、HDL 検証における小規模 validation input 生成として手法を定義する。問題依存性の知見を踏まえると、Hybrid-UGT は「全 Verilog 候補を対象とする万能手法」ではなく、「特定の構文構造を含む候補について追加 validation input を設計する道具」として位置付けるのが妥当である。

8. 本研究の制約

第一に、Hybrid-UGT は failure detection 手法であり、正しさを証明する手法ではない。生成 validation input vector で不一致が出た候補は失敗と判定できるが、不一致が出ない候補が正しいとは限らない。したがって、実運用では公式 testbench、商用シミュレータ、形式検証、合成後検証などの full validation が依然として必要である。

第二に、UGT 評価は、正式生成済み候補のうち compile-pass official-fail 候補を抽出したオフライン評価である。この設定は failure detection 能力を分離して測るためには有用だが、候補集合全体に対する最終成功率や開発時間削減を直接示すものではない。評価対象は、全 CPOF 34,805 件のうち生成 token 数 ≤ 512 の 31,842 件 (91.5%) であり、残り 3,059 件 (8.5%) は prompt logprobs 再スコアリングのメモリ制約から除外している。特に長い候補では Hybrid-UGT の効果が変わる可能性がある。

第三に、本稿では VerilogEval の参照実装を用いて生成 validation input vector の出力比較を行った。実際の開発では常に参照実装が存在するとは限らない。その場合は、仕様 assertion、既存 golden model、プロパティチェック、または高信頼な simulation model を oracle として使う必

要がある。

第四に、主評価は Qwen3.5-9B-AWQ-4bit, temperature=0.8, $k = 10$ に固定している。モデル、量子化方式、デコーディング設定が変われば、token log 確率の分布や低確信度 span の意味も変わり得る。また、prompt logprobs を再スコアリングで取得する場合、追加の推論コストとメモリコストが発生する。

第五に、本稿の Random ベースラインは bit 幅に応じた一様サンプリングである。これは産業的 Verilog 検証で広く用いられる constrained random verification (CRV) や開発者が hand-craft する directed testbench を代替しない。本稿の Hybrid-UGT 評価は「uniform random との比較における機構的検出効率」を測ったものであり、CRV ベースとの定量比較や Hybrid-UGT の CRV への統合は今後の課題である。特に、counter 系 +0.145 det@1 や shift 系 -0.061 det@16 といった効果量は本稿の評価設定における値であり、CRV が対象とするより構造化された入力分布の下での挙動は別途検証を要する。

第六に、本稿の評価結果は、本稿で git 管理下に再実装したパイプライン (span 抽出, 5 手法のベクトル生成器, テストベンチテンプレート, iverilog/vvp ドライバ) に依存する。低 log 確率 token から span を切り出して焦点信号や境界値を抽出するロジック, および順序回路における各 vector 後の fill cycle 数 (本稿では 8 サイクル) といった実装詳細は、過去文献の記述だけでは一意に確定しない。従って tag 別の効果量は実装が異なれば変化し得る。本稿は、論文の主張を「specific aggregate detection rate」ではなく「論文中の手法仕様にに基づく再現可能な reference implementation における挙動」として位置付ける。

9. 結論

本稿では、LLM による Verilog コード生成候補に対し、token log 確率を用いた不確実性誘導テストを提案・実装・評価した。まず、VerilogEval 156 問・156,000 候補を用いて Mean, Tail, MinLogP による candidate ranking を評価し、log 確率集約スコアはコンパイル失敗候補の後回しには有効だが、コンパイル通過後の semantic correctness 識別には限定的であることを示した。

この限界を踏まえ、低 log 確率 token の位置を局所的不確実性 span として扱い、span 内の信号名, bit 範囲, 演算子, case/state, reset/clock から小規模 directed validation input を生成する Hybrid-UGT を提案した。本稿では、サブサンプリング選定, prompt logprobs 再スコアリング, span 特徴抽出, 5 手法のベクトル生成器, testbench 生成, シミュレータドライバ, 並列 orchestrator からなる評価パイプライン全体を git 管理下に再実装した。

compile-pass official-fail 候補のうち生成 token 数 ≤ 512 の全件 31,842 候補に対する評価では、Hybrid-UGT が de-

tection budget の小さい領域で Random を明確に上回ることが示された。全体で det@1 は 0.524 (Random 0.496, $\Delta = +0.028$), counter 系 ($n=2,375$) で +0.145 det@1, posedge clock 系 ($n=13,867$) で +0.068 det@1, FSM/state 遷移系 ($n=3,889$) で +0.058 det@1 の優位を示した。一方, det@16 では Hybrid-UGT 0.841, Random 0.845 と同等に収束し ($\Delta = -0.004$), shift/rotate 系 ($n=3,139$) のみ -0.061 det@16 の顕著な劣後が残る。

この性能曲線は、Hybrid-UGT を「少数 validation vector で明らかな fail 候補を fail-fast 棄却する」per-request screening の用途に位置付けることを支持する。Random uniform は B を増やせば最終的に同等の累積検出率に達するが、1~4 ベクトル目で多くの失敗を踏み落とす Hybrid-UGT の方が、screening コスト最小化の観点では有利である。今後は、(i) span 抽出ロジックの強化 (output 駆動の挙動や状態変数を間接的に動かす入力 of 自動推定) による shift/rotate 系の det@16 改善, (ii) constrained random verification との統合・比較, (iii) より多くの Verilog 構造 (テストベンチ依存の入力タイミング, async reset, clock-domain crossing) への手法拡張, (iv) 別モデル・別ベンチマーク (VerilogEval-Specification, RTL-Coder) での汎化性検証, が必要である。

参考文献

- [1] Chen, M. et al.: Evaluating Large Language Models Trained on Code, Technical Report arXiv:2107.03374, OpenAI (2021).
- [2] Li, Y. et al.: Competition-level code generation with AlphaCode, *Science*, Vol. 378, No. 6624, pp. 1092–1097 (online), DOI: 10.1126/science.abq1158 (2022).
- [3] Wang, X. et al.: Self-Consistency Improves Chain of Thought Reasoning in Language Models, *Proc. 11th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=1PL1NIMMrw> (2023).
- [4] Chen, B. et al.: CodeT: Code Generation with Generated Tests, *Proc. 11th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=ktrw68Cmu9c> (2023).
- [5] Le, H. et al.: CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning, *Advances in Neural Information Processing Systems 35 (NeurIPS)*, (online), available from <https://arxiv.org/abs/2207.01780> (2022).
- [6] Farquhar, S. et al.: Detecting hallucinations in large language models using semantic entropy, *Nature*, Vol. 630, No. 8017, pp. 625–630 (online), DOI: 10.1038/s41586-024-07421-0 (2024).
- [7] Zhang, H. et al.: Self-Improving Code Generation via Semantic Entropy and Behavioral Consensus, Technical Report arXiv:2603.29292, arXiv (2026).
- [8] Zhang, T. et al.: TokUR: Token-Level Uncertainty Estimation for Large Language Model Reasoning, *Proc. 14th International Conference on Learning*

- Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=VHQc7wzmYv> (2026).
- [9] Liu, T. et al.: Token-Level Uncertainty-Aware Objective for Language Model Post-Training, Technical Report arXiv:2503.16511, arXiv (2025).
 - [10] Wang, S. et al.: USCD: Improving Code Generation of LLMs by Uncertainty-Aware Selective Contrastive Decoding, Technical Report arXiv:2409.05923, arXiv (2024).
 - [11] Sharma, A. and David, C.: Assessing Correctness in LLM-Based Code Generation via Uncertainty Estimation, Technical Report arXiv:2502.11620, arXiv (2025).
 - [12] Xu, S. et al.: Uncertainty Quantification for LLM-based Code Generation, Technical Report arXiv:2605.12201, arXiv (2026).
 - [13] He, K. et al.: Towards Better Code Generation: Adaptive Decoding with Uncertainty Guidance, Technical Report arXiv:2506.08980, arXiv (2025).
 - [14] Zhu, Y. et al.: Hot or Cold? Adaptive Temperature Sampling for Code Generation with Large Language Models, *Proc. AAAI Conference on Artificial Intelligence (AAAI)*, (online), available from <https://arxiv.org/abs/2309.02772> (2024).
 - [15] Zhang, S. et al.: EDT: Improving Large Language Models' Generation by Entropy-based Dynamic Temperature Sampling, Technical Report arXiv:2403.14541, arXiv (2024).
 - [16] Madaan, A. et al.: Self-Refine: Iterative Refinement with Self-Feedback, *Advances in Neural Information Processing Systems 36 (NeurIPS)*, (online), available from <https://arxiv.org/abs/2303.17651> (2023).
 - [17] Liu, M. et al.: VerilogEval: Evaluating Large Language Models for Verilog Code Generation (2023).
 - [18] Kwon, W. et al.: Efficient Memory Management for Large Language Model Serving with PagedAttention, *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626 (online), available from <https://arxiv.org/abs/2309.06180> (2023).